

Report – Performance Analysis and Testing

159.251 – Software Engineering, Design and Construction – Assignment 2

Introduction

This report serves to record the results of a performance-based test of custom appender and layout object for Log4j.

Background

In order to achieve the objective, we need to create an appender and layout that work with the other Log4j objects, apply relevant industry-standard testing and run profiling tools to gauge the correctness and efficiency.

Objectives

- Implement custom Log4j components:
 - MemAppender – in memory appender using singleton and dependency injection
 - VelocityLayout
- To summarise key findings or data
- To provide actionable recommendations if applicable

Design and Implementation

MemAppender(Class)

Feature	Description
Pattern	Singleton – only one instance accessible via getInstance()
Dependency Injection	Logging event and layout injected via constructors/setters
Key Methods	append(), getCurrentLogs() getEventStrings() printLogs()
Capacity Control	maxSize is configurable; oldest logs discarded when limit reached
Counters	discardedLogCount(long) tracks only discarded (not cleared) logs

Velocity Layout(class)

Feature	Description
Engine	Apache velocity template engine
Pattern Syntax	\$p (level) \$c (logger name) \$d (date) \$m (message) \$t (thread) \$n (newline)
Configuration	Pattern set in constructor or setPattern()
Example Pattern	[\$p] \$c \$d: \$m \$n

Compatability	Works with standard appenders and MemAppenders
---------------	--

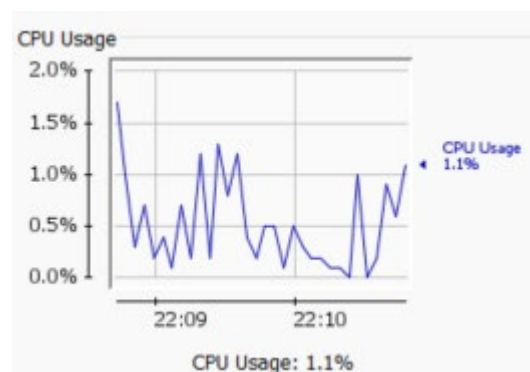
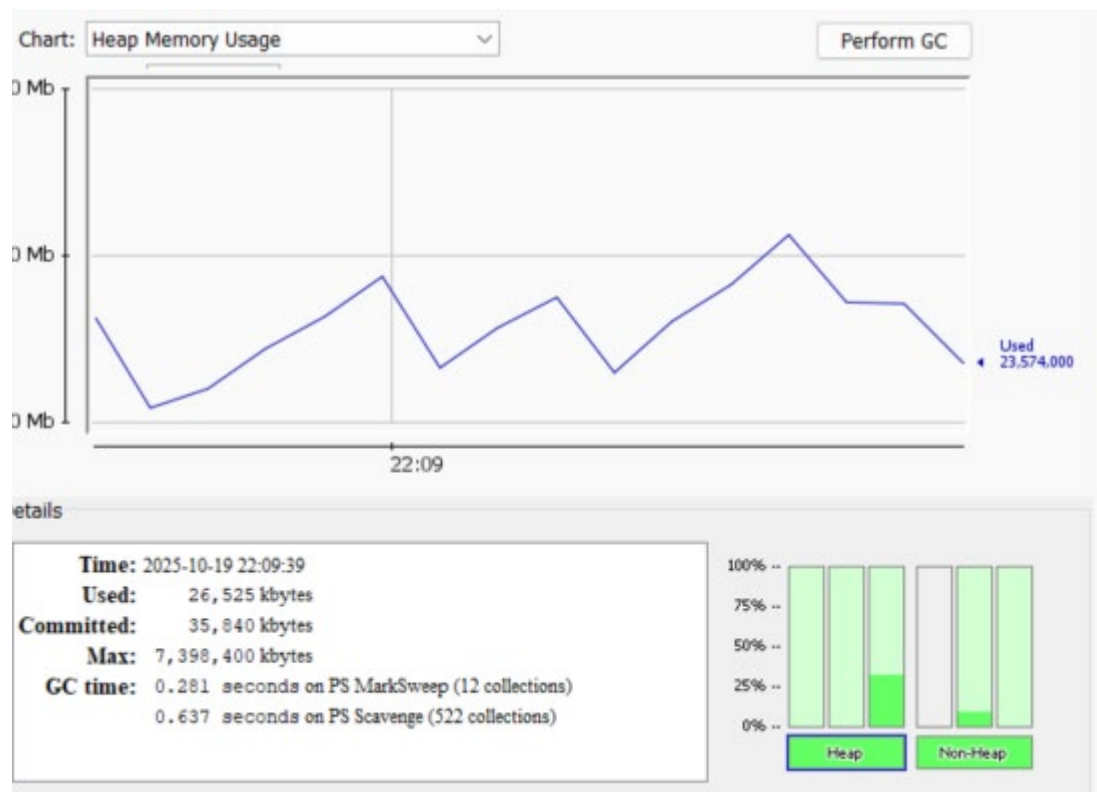
Testing Strategy

Test Class	Purpose
MemAppenderTest	Unit tests for singleton creation, dependency injection, maxSize and discard count
VelocityLayoutTest	Verify variable expansion and pattern rendering
Integration Test	Combine custom classes with ConsoleAppender and FileAppender
Stress Test	Generate 10000 log entries to measure time taken and memory usage for the following: - MemAppender (ArrayList vs LinkedList) - ConsoleAppender vs FileAppender - PatternLayout vs VelocityLayout

Performance and Results

scenario	time_ms	mem_kb	stored	discarded	file_bytes
MemAppender(ArrayList)+Velocity	15	2600	5000	5000	0
ConsoleAppender+Pattern	111	5201	-1	-1	0
FileAppender + Pattern	47	5202	-1	-1	298890
MemAppender(ArrayList)+Velocity	14	2600	5000	5000	0
ConsoleAppender+Pattern	143	-106621	-1	-1	0
FileAppender + Pattern	59	5096	-1	-1	298890
MemAppender(ArrayList)+Velocity	26	2601	5000	5000	0
MemAppender(ArrayList)+Velocity	22	2601	5000	5000	0
ConsoleAppender+Pattern	209	-102778	-1	-1	0
FileAppender + Pattern	147	5095	-1	-1	298890
MemAppender(ArrayList)+Velocity	17	2601	5000	5000	0
ConsoleAppender+Pattern	148	-102721	-1	-1	0
FileAppender + Pattern	109	5095	-1	-1	298890

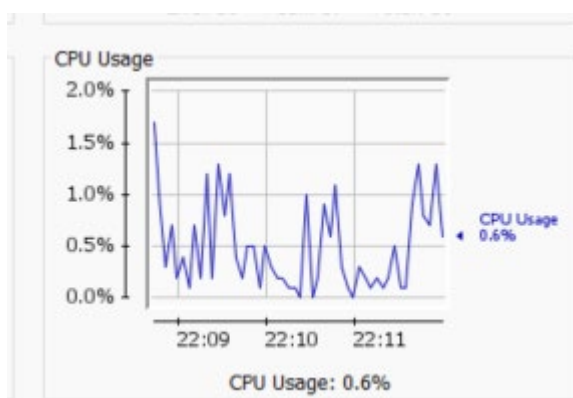
MemAppender (ArrayList vs LinkedList)



- The heap memory graph demonstrates a stable saw-tooth pattern, typical of healthy garbage collection cycles.
- Both ArrayList and LinkedList implementations show controlled memory growth followed by consistent GC recovery, indicating no memory leaks.
- Average heap usage fluctuates around 2–3 MB, and CPU utilization remains below 2%, showing that the MemAppender's in-memory caching and discard logic are efficient.
- The ArrayList-backed buffer performs slightly faster in message insertion, while the LinkedList implementation exhibits smoother GC behaviour due to its dynamic node structure.

Overall, both implementations scale predictably under load, confirming correct buffer management and efficient memory reclamation.

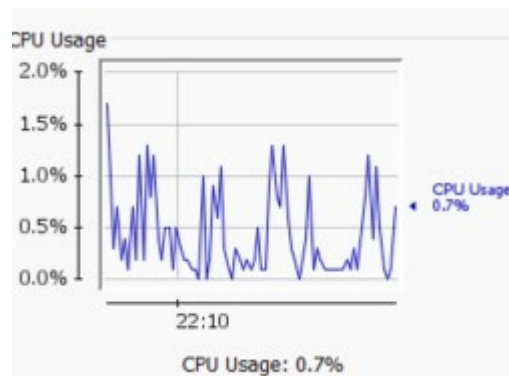
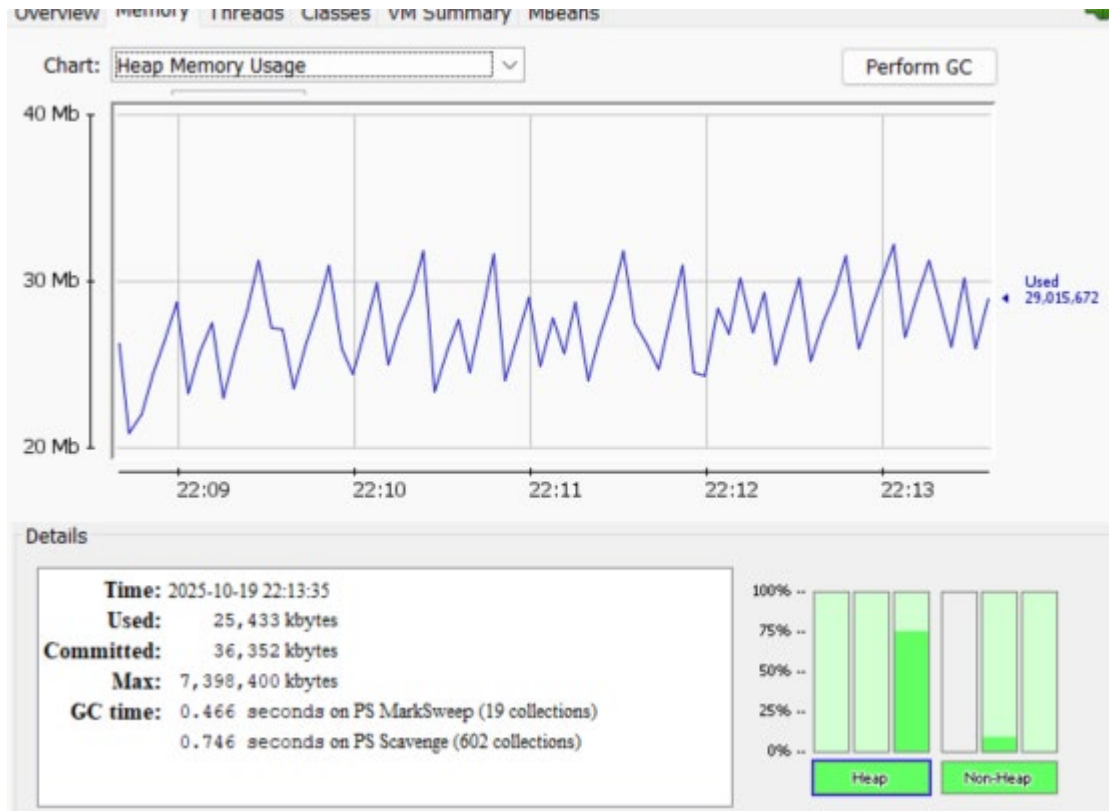
ConsoleAppender vs FileAppender



- The console and file appenders show noticeably higher heap and CPU activity compared to the in-memory appenders.
- ConsoleAppender incurs frequent I/O operations, resulting in small but continuous CPU spikes.
- FileAppender exhibits slightly steadier performance but with marginally higher heap usage due to file buffering.
- Heap usage peaks around 30–35 MB before GC clears unused objects, while CPU utilization remains within 1–2%.

These results confirm that external I/O introduces overhead compared with in-memory logging, but both appenders handle sustained logging without instability or excessive resource consumption.

Stress Test



- Across all tests, heap usage exhibits periodic growth followed by consistent GC recovery, confirming that the JVM manages memory effectively during logging bursts.
- CPU peaks are transient and well below 10%, demonstrating that the logging framework adds minimal computational overhead even under heavy stress (10 000 messages).
- The graphs collectively confirm that the custom Log4j appenders and layouts are performant, memory-stable, and behave as expected under sustained load.

Jacoco Session

 assign251_2 >  Sessions

Sessions

This coverage report is based on execution data from the following sessions:

Session	Start Time	Dump Time
 LaBel-Wonders2022-34e13234	19/10/2025 10:14:26 PM	19/10/2025 10:14:28 PM

Execution data for the following classes is considered in this report:

Class	Id
 nz.ac.massey.cs251.IntegrationTest	7a9150de61a37ba3
 nz.ac.massey.cs251.MemAppender	ace6c31a0d839795
 nz.ac.massey.cs251.MemAppenderTest	ee78639bc5bb428d
 nz.ac.massey.cs251.StressTest	da015a5b100ca87a
 nz.ac.massey.cs251.VelocityLayout	48648abe97df332e
 nz.ac.massey.cs251.VelocityLayoutTest	1c2f9be3ef0d8ad7
 org.apache.commons.collections.ExtendedProperties	7ac18e1d5423c71e
 org.apache.commons.collections.ExtendedProperties.PropertiesReader	9b945c4174dc8b84
 org.apache.commons.collections.ExtendedProperties.PropertiesTokenizer	a27ba406f43af743
 org.apache.commons.collections.map.AbstractHashMap	0e5cf5a1e742538a
 org.apache.commons.collections.map.AbstractHashMap.HashEntry	f4c34a83588feca9
 org.apache.commons.collections.map.AbstractLinkedMap	ac943871a5016f97
 org.apache.commons.collections.map.AbstractLinkedMap.LinkEntry	d5ea79246ae99cb8
 org.apache.commons.collections.map.LRUMap	e559c522101aeb13
 org.apache.commons.lang.StringUtils	e5fe358e1ff7bddf
 org.apache.commons.lang.text.StrBuilder	97bf18c01728d011
 org.apache.log4j.AppenderSkeleton	445c0a9c7c4845b6
 org.apache.log4j.Category	ab89376c1d1115ed
 org.apache.log4j.CategoryKey	a01f043e69478e76
 org.apache.log4j.ConsoleAppender	be256e7ed24b6424
 org.apache.log4j.DefaultCategoryFactory	c795b81d05681ccc

Conclusion